

category: avalonia

tags: [avalonia, camera, timing, layout, motion-detection, startup, dispatcher]

severity: critical

created: 2026-07-18

updated: 2026-07-18

project-origin: parking-v8-app-avalonia

Avalonia `AttachedToVisualTree` fires trước layout → `Bounds = 0` → camera decode full native resolution

Tình huống gặp phải

Đang điều tra BUG-004: `MainShellWindow` "Not Responding" ngay sau đăng nhập trong scenario multi-lane (2 làn × 3 camera = 6 camera cùng khởi động), **chỉ xảy ra khi Motion detection bật**.

Môi trường: Avalonia UI (.NET 8, Windows 11 x64), `parking-v8-app-avalonia`, `EntryLaneCameraTileView.axaml.cs`.

Triệu chứng / Lỗi

```
MainShellWindow: "(Not Responding)" ngay sau khi hiện lên
CPU: 8+ cores liên tục dù UI idle (không có sự kiện xe)
RAM: tăng từ ~180MB lên 1237MB+ trong vài giây
App crash sau ~35-55 giây
```

Đặc biệt: triệu chứng **KHÔNG** xuất hiện khi:

- Motion detection tắt (dù cùng multi-lane scenario)
- Chỉ 1 camera đơn lẻ với motion detection bật

Nguyên nhân gốc rễ (Root Cause)

Ba điều kiện phải xảy ra đồng thời:

1. ``AttachedToVisualTree`` → ``StartCamera()`` ngay lập tức: Event này fires **trước** khi Avalonia hoàn tất Measure/Arrange pass đầu tiên. Tại thời điểm đó `Bounds.Width = 0`.
2. ``_cachedWidth = 0``: `AnvPlayer` dùng volatile field `_cachedWidth/_cachedHeight`, chỉ được cập nhật từ `OnPropertyChanged(BoundsProperty)` khi `Width > 0`. Vì layout chưa xong, event này chưa fire với giá trị hợp lệ → `_cachedWidth = 0` khi `StartCamera()` chạy.
3. **Decode tại native resolution thay vì control size**:
 - `controlReady = (_cachedWidth >= 32) = (0 >= 32) = false`
 - → decode frame tại native camera resolution (1920×1080) thay vì control size (~300px)
 - → 41× nhiều pixel hơn cần thiết
 - `motionDetectionInterval = 0` → mọi frame đều qua OpenCV pipeline (`ProcessFrame()`):
Absdiff + Threshold + Erode)

- → 6 cameras × 30fps × nặng OpenCV → CPU bị chiếm hết → UI thread starved → "Not Responding"

```
AttachedToVisualTree fire (trước Measure/Arrange)
→ StartCamera() gọi ngay
→ _cachedWidth = 0 (layout chưa done)
→ controlReady = false
→ decode at 1920×1080 (native res, không phải ~300px control size)
→ motionDetectionInterval=0: MỖI frame qua OpenCV
→ 6 cameras × 30fps × ProcessFrame()
→ CPU 8+ cores → UI starved → Not Responding
```

Giải pháp

Fix 1 — Delay StartCamera() đến sau layout pass (BẮT BUỘC)

```
// TRƯỚC (crash): AttachedToVisualTree fires trước layout → Bounds.Width = 0
AttachedToVisualTree += (_, _) => StartCamera();

// SAU (fix): DispatcherPriority.Loaded fires SAU khi layout pass hoàn tất
// → Bounds.Width có giá trị hợp lệ → _cachedWidth >= 300 → decode đúng kích thước
AttachedToVisualTree += (_, _) => Dispatcher.UIThread.Post(StartCamera,
DispatcherPriority.Loaded);
```

`DispatcherPriority.Loaded` được Avalonia đảm bảo chạy sau layout pass — đây là cách chuẩn để delay code đến khi control đã có kích thước thật.

Fix 2 — motionDetectionInterval 0 → 100ms

```
// TRƯỚC (mọi frame qua OpenCV):
camera.Start(fromMotion, toMotion, 0, false, aiBoxes, false, loopType);

// SAU (tối đa 10 motion check/giây):
camera.Start(fromMotion, toMotion, 100, false, aiBoxes, false, loopType);
```

`motionDetectionInterval = 0` = mọi frame qua OpenCV, không có giới hạn. Với 30fps × 6 cameras, đây là tải không cần thiết. `100ms` = tối đa 10 check/giây, giảm tải OpenCV 3×.

Fix 3 — CloseVideoSource() luôn gọi Stop()

```
// TRƯỚC (bỏ sót camera chưa kết nối):
if (videoSourcePlayer != null && videoSourcePlayer.IsRunning)
    videoSourcePlayer.Stop();

// SAU (luôn release native memory):
if (videoSourcePlayer != null)
    videoSourcePlayer.Stop(); // BUG-004 Fix 3
```

Camera chưa kết nối có `IsRunning = false` → `Stop()` bị bypass → native FFmpeg memory không được release.

Fix 4-6 — Các vấn đề phụ trong SDK

```
// Fix 4: OperationCanceledException riêng trong retry loop
catch (OperationCanceledException) when (token.IsCancellationRequested)
{
    return; // Thoát sạch, không exception storm
}

// Fix 5: VideoStreamDecoderIntptr.Dispose() phải complete channels
```

```
FrameChannel.Writer.TryComplete();  
MdChannel.Writer.TryComplete();  
AiChannel.Writer.TryComplete();  
  
// Fix 6: Dispose kernel Mat trong FrameMotionDetector.ProcessFrame()  
using var erodeKernel = new Mat(); // Thay new Mat() không dispose  
Cv2.Erode(diff, diff, erodeKernel);
```

Áp dụng lại (How to reuse)

- Khi thấy crash/freeze chỉ với **một combination cụ thể** (VD: motion detection BẬT + multi-camera) → nghi ngờ tổ hợp nguyên nhân, không phải từng nguyên nhân riêng lẻ
- Khi camera decode tốn tài nguyên không ngờ → kiểm tra `_cachedWidth/_cachedHeight`: có thể = 0 nếu `StartCamera()` gọi trước layout
- Bất cứ khi nào dùng `AttachedToVisualTree` để trigger logic cần `Bounds` hợp lệ → PHẢI dùng `Dispatcher.UIThread.Post(..., DispatcherPriority.Loaded)` thay vì gọi trực tiếp
- Nếu `motionDetectionInterval = 0` → mọi frame qua OpenCV → CPU cao không cần thiết; luôn set `>= 100ms` cho production

Chú ý / Cạm bẫy (Gotchas)

- ⚠ **Không phải mọi kết hợp đều crash**: Motion detection tắt + size=0 → fine; size=0 tắt + motion on → fine; chỉ CÙNG LÚC mới crash → dễ nhầm nguyên nhân
- ⚠ `_cachedWidth = 0` im lặng`: Không có exception, không có log — `controlReady = false` nên code chạy "bình thường" nhưng dùng resolution sai
- ⚠ `DispatcherPriority.Loaded` ≠ DispatcherPriority.Background``: `Loaded` được đảm bảo sau layout; `Background` chỉ là low priority, không đảm bảo layout đã xong
- ⚠ `motionDetectionInterval = 0` là hợp lệ theo API` nhưng có nghĩa "không giới hạn" — tài liệu không cảnh báo về tác động CPU
- ⚠ `ReadAllAsync` block mãi nếu không TryComplete()`: Consumer tasks stuck sau decoder replacement — phải complete channels trong `Dispose()`

Tham chiếu

- BUG-004: [docs/bugs/BUG-004-mainshell-freeze-after-login.md](#)
- GOTCHAS.md G008 (project-level)
- Avalonia Dispatcher docs: <https://docs.avaloniaui.net/docs/concepts/the-main-window/dispatcherpriority>
- Project: [parking-v8-app-avalonia](#), session 2026-07-18